Code Explanation

Code Explanation: AWS Glue PySpark ETL Job

Overview

This is a production-grade AWS Glue PySpark job that implements a **Customer Order Analytics pipeline** with **SCD Type 2** (Slowly Changing Dimension) processing using Apache Hudi.---

High-Level Flow

1. **Ingest** customer and order data from S3 curated zones2. **Clean** data (remove NULLs, duplicates, invalid strings)3. **Apply SCD Type 2** columns (IsActive, StartDate, EndDate, OpTs)4. **Calculate** customer aggregate spend metrics5. **Write** results to S3 using Apache Hudi format6. **Register** tables in AWS Glue Catalog---

Component Breakdown

1. **SparkContextManager**

Purpose: Manages Spark/Glue context lifecycle**Key Methods:**- `initialize_spark_contexts()`: Creates SparkContext, GlueContext, SparkSession with optimized configurations (adaptive execution, Kryo serialization)- `initialize_glue_job()`: Initializes AWS Glue Job with resolved parameters- `cleanup()`: Commits job and stops Spark session---

2. **S3DataManager**

Purpose: Handles all S3 read/write operations with validation**Key Methods:****`validate_s3_path(s3_path)`**- Parses S3 URI (s3://bucket/prefix)- Checks bucket accessibility via `head_bucket()`- Verifies objects exist via `list_objects_v2()`**`read_data_safe(s3_path, file_format, schema)`**- Validates path before reading- Reads data (Parquet/CSV/JSON)- Normalizes column names to lowercase- Returns DataFrame or None on failure**`write_data_safe(df, s3_path, file_format, mode, partition_by)`**- Writes DataFrame to S3- Supports partitioning- Logs record counts**`write_hudi_safe(df, s3_path, table_name, record_key, precombine_key, operation)`**- Writes data using Apache Hudi format- Configures Hudi options: - `COPY_ON_WRITE` table type - Record key for primary key - Precombine key for deduplication - Cleaner policy (keeps last 10 commits)- Supports upsert/insert/bulk_insert operations---

3. **DataCleaner**

Purpose: Implements data quality rules**Key Methods:****`remove_nulls(df, required_fields)`**- Filters out rows where required fields are NULL- Logs count of removed rows**`remove_null_strings(df, null_values)`**- Replaces string values like "Null", "null" with actual NULL- Applies to all StringType columns- Case-insensitive matching**`remove_duplicates(df, key_columns, order_column)`**- Uses window function with `row_number()`- Partitions by key columns- Orders by timestamp descending- Keeps only the latest record (row_num = 1)---

4. **SCD2Processor**

Purpose: Implements Slowly Changing Dimension Type 2 logic**Key Methods:****`add_scd2_columns(df)`**Adds four columns:- `IsActive`: Boolean (True for new records)- `StartDate`: Current timestamp (record effective start)- `EndDate`: NULL (record not yet expired)- `OpTs`: Current timestamp (operation timestamp for versioning)**`validate_scd2_columns(df)`**- Verifies all SCD Type 2 columns exist- Returns boolean validation result---

5. **CustomerAggregator**

Purpose: Calculates customer-level metrics**Key Method:****`calculate_customer_aggregate_spend(customer_df, order_df)`**1. **Joins** customer and order data on `customer_id`2. **Groups by** customer_id, customer_name3. **Calculates aggregates:** - `total_orders`: Count of orders - `total_spend`: Sum of order amounts - `avg_order_value`: Average order amount - `first_order_date`: Earliest order date - `last_order_date`: Most recent order date4. **Casts** decimal columns to `DecimalType(20, 2)`---

6. **GlueCatalogManager**

Purpose: Manages AWS Glue Data Catalog operations**Key Methods:****`create_database_if_not_exists()`**- Checks if database exists via `get_database()`- Creates database if not found- Idempotent operation**`register_table(table_name, s3_path, columns)`**- Defines storage descriptor (Parquet format, Hive SerDe)- Updates existing table or creates new one- Registers table metadata in Glue Catalog---

7. **CustomerOrderETLJob**

Purpose: Main orchestrator class**Initialization:**- `_load_config(config_path)`: Loads YAML config from S3 or local file- `initialize()`: Sets up all managers and creates Glue database**Pipeline Stages:****Stage 1: `ingest_data()`**- Reads customer data from `s3://adif-sdlc/curated/sdlc_wizard/customer/`- Reads order data from `s3://adif-sdlc/curated/sdlc_wizard/order/`- Returns tuple of DataFrames**Stage 2: `clean_data(customer_df, order_df)`**- Applies cleaning rules from config: - Remove null strings ("Null", "null", "NULL") - Remove rows with NULL in required fields - Remove duplicates keeping latest record- Returns cleaned DataFrames**Stage 3: `process_scd2(customer_df, order_df)`**- Adds SCD Type 2 columns to both DataFrames- Validates column presence- Returns SCD2-enabled DataFrames**Stage 4: `calculate_aggregations(customer_df, order_df)`**- Calculates customer aggregate spend metrics- Adds SCD Type 2 columns to aggregation result- Returns aggregate DataFrame**Stage 5: `write_results(order_df, customer_agg_df)`**- Writes order summary to `s3://adif-sdlc/analytics/ordersummary/`- Writes customer aggregate to `s3://adif-sdlc/analytics/customeraggregatespend/`- Uses Hudi format with upsert operation**Stage 6: `register_catalog_tables()`**- Registers `customer_aggregate_spend` table in Glue Catalog- Defines schema with 11 columns (including SCD Type 2 fields)**`run()`**- Executes all 6 stages sequentially- Logs progress with stage headers- Returns success/failure boolean- Ensures cleanup in finally block---

8. **main() Function**

Entry Point:1. Retrieves `config_path` from Glue job arguments2. Creates `CustomerOrderETLJob` instance3. Executes `job.run()`4. Exits with status code (0 = success, 1 = failure)---

Key Technical Features

Error Handling

- Try-catch blocks in all methods- Detailed logging at each step- Safe returns (None/False) on failure- Cleanup guaranteed via finally block

Data Quality

- NULL removal (actual NULL and string "Null")- Deduplication with latest record selection- Required field validation- Column name normalization (lowercase)

Performance Optimizations

- Spark adaptive execution enabled- Partition coalescing enabled- Kryo serialization- Window functions for deduplication

Apache Hudi Integration

- COPY_ON_WRITE table type- Record-level upserts- Automatic deduplication- Commit retention (10 commits)

AWS Integration

- Glue Catalog registration- S3 path validation- Boto3 for AWS API calls- YAML config from S3---

Configuration Requirements

The job expects a YAML config with:- `source_paths`: Customer and order S3 locations- `target_paths`: Output S3 locations- `data_cleaning`: Cleaning rules and required fields- `hudi_config`: Table names, keys, operation type- `glue_catalog`: Database name---

Output

Two Hudi Tables:1. **Order Summary** (with SCD Type 2)2. **Customer Aggregate Spend** (with metrics + SCD Type 2)**Glue Catalog:**- Registered table: `customer_aggregate_spend`- Database: `sdlc_wizard_db`